SchoolApp360 Project Audit Report

Admin web panel and Expo mobile app: security, architecture, data, performance, operations

Generated 16 October 2026
Read-only audit — no application source code was modified.
Confidential: contains security findings. Contains no credentials or personal data.

Contents

SchoolApp360 - Project Audit Report

1. Executive summary
2. Audit scope and methodology
3. Architecture overview
4. Findings overview by theme
5. Detailed findings register
6. Missing mandatory requirements
7. Security assessment
8. Functional and technical audit results
9. Testing and verification summary
10. Prior audit reconciliation
11. Remediation roadmap
12. Final recommendations
13. Not verified and limitations
14. Glossary
15. Report files

SchoolApp360 - Project Audit Report

Audit date: 16 October 2026. Scope: admin web panel (Next.js 16, Prisma/Postgres) and the Expo mobile app. Read-only audit; no application source, database or configuration was changed.

1. Executive summary

Readiness classification: Not Ready for Production.

The audit produced 144 finding entries (about 125 distinct issues once duplicates reported by more than one auditor are merged): P0: 1, P1: 24, P2: 56, P3: 53, P4: 10. Of the 21 mandatory requirements assessed, 21 are not fully implemented (6 failed, 4 partial, 9 missing, 1 failed (suspected), 1 not verified).

Why this classification. The most serious problems sit in the layer that protects every school from every other school:

- Requests without a valid session are treated as a built-in "dashboard Teacher" actor, and some write paths and a cross-school support-ticket read are reachable that way (API-001, SEC-017).
- The active school is a process-global singleton that every login changes, and an unauthenticated route can repoint it (SEC-004, API-002).
- Five handlers, including the fee ledger and the Razorpay create-order and verify endpoints, have no authentication check (API-003).
- Any signed-in Parent or Accountant can write homework and exam marks and read full student records (SEC-001, PRV-001).
- Online payments are confirmed only when the browser calls back; there is no webhook, collection has race conditions, and the fee cron serves one school and fails open without its secret (BIZ-002, BIZ-003, DB-002, BIZ-001, OBS-004).
- The framework version in use has critical advisories (SEC-008); identity documents sit under public URLs (ARC-001, PRV-002).
- There are no automated tests, no CI, and the migration history covers 2 of 52 models (TST-001, OPS-001, DB-001).

Security posture. Injection classes are handled well (parameterised Prisma, escaped print HTML, no `eval` / `innerHTML`), passwords use bcrypt, per-school payment secrets are AES-GCM encrypted, and Razorpay signatures use constant-time comparison. Access control, tenant isolation, session lifecycle, headers and dependency hygiene are weak. This is not a certification of security or compliance; no dynamic testing was possible because the dev server was not running.

Technical debt. ESLint reports 97 errors and 35 warnings (47 setState-in-effect, 42 unescaped entities); `lib/api.js` is 2,181 lines and `lib/fees.js` 1,728; a legacy role model and a newer `roleKey` model coexist; class, section and subject are plain strings in most tables; several pages and jobs still load whole rosters.

What is in good shape. 101 of 180 routes use permission checks; parent and teacher mobile routes are scoped by session; server-side pagination exists on the main lists; the import graph of 703 files has no static cycles; 24 of the 67 items from the earlier 42/100 audit were re-verified as fixed (unauthenticated mutation routes named there now carry guards).

Recommended next actions. Do Phase 0 first (section 11): remove the fallback actor and global singleton, guard the unprotected handlers, make the crons fail closed, upgrade `next`. These are mostly small changes with the highest risk reduction. Then Phase 1 (authorization, payments, documents, migrations, tests and CI).

If real schools are already using the deployed system, treat Phase 0 as an incident-level priority.

Unverified areas

Production hosting settings (Vercel, environment values including `CRON_SECRET`), the object-storage bucket policy, database backups, Pusher channel authorisation, secret strength, runtime performance, bundle size and live behaviour of any endpoint were not verified. See section 13.

Scorecard

Scores are 0 to 5 as given by each auditor (5 needs substantial verification). Areas whose auditor stopped before exporting scores are shown as not scored; their findings are still in the register.

Area	Score	Confidence	Basis
Testing and QA	1	High	No app tests in ADMIN or MOBILE; eslint exits 1 with 97 errors.
DevOps and deployment	2	Medium	Vercel config and env-based secrets exist; no CI, no env validation, no migration workflow, cron fails open.
Maintainability	3	Medium	No unused deps, zero TODOs, layered lib/; but 2k-line modules, boilerplate README, red lint.
Frontend engineering	3	Medium	No HTML sinks, escaped print output, stale-safe paged hook; but no error boundaries and 55 eslint errors.
UI/UX	3	Low	Code-based only: skeleton loaders in redesigned areas, but Modal/Dropdown lack a11y and loaders/pagination are inconsistent.
Mobile application	2	Medium	Demo credentials shown in production login, no 401 handling, app source untracked in git; SecureStore and idempotent-only retry are good.
Authentication	2	Medium	bcrypt, httpOnly Secure cookie, generic login errors and rate limiting exist; but no revocation, in-memory limiter, weak reset-token handli…
Authorization and tenant isolation	1	High	29 of 180 routes only check actor non-null; Parent can write homework/marks; global school singleton; global roles.
Application security	2	Medium	Print HTML escaped, SQL parameterised, Razorpay HMAC with timingSafeEqual and AES-GCM secrets; no headers, vulnerable next version.
Performance	3	Medium	Paged and aggregated paths exist for the main lists; class pages, fee generation and dashboard aggregates still load whole tables (lib/clas…
Scalability	2	Medium	No pool configuration in lib/db.js; bulk import, promotion and fee generation are per-row sequential work; cron serves one school.

Area	Score	Confidence	Basis
Maintainability	3	Medium	No import cycles in 703 files and lib never imports UI, but lib/api.js is 2,181 lines and helpers are duplicated (formatCurrency in 19 file...
Backend and APIs	not scored	-	Auditor stopped before exporting scores; see the findings register (Backend/API, Database, Observability/Privacy entries)
Database design and integrity	not scored	-	Auditor stopped before exporting scores; see the findings register (Backend/API, Database, Observability/Privacy entries)
Observability	not scored	-	Auditor stopped before exporting scores; see the findings register (Backend/API, Database, Observability/Privacy entries)
Privacy and operational readiness	not scored	-	Auditor stopped before exporting scores; see the findings register (Backend/API, Database, Observability/Privacy entries)

2. Audit scope and methodology

Eleven audit streams worked read-only on both repositories: repository and architecture, frontend and UX, backend and API, security and tenant isolation, database, performance, mobile app, testing, DevOps, observability and privacy, and reconciliation of the earlier audit. Findings were traced from UI to API to library to Prisma where possible, and every earlier claim was re-opened in the current tree rather than copied.

Rules followed: no source edits, no database writes, no secret values read or printed, no destructive commands, no tests against production. Confidence is stated per finding; "Suspected" means code suggests a defect that was not confirmed.

Timing note: the first group of streams ran on 10 October 2026 and the second group (security, performance, testing and DevOps) on 16 October 2026. The working tree changed in between (for example the lint error count moved from 55 to 97); the later streams re-verified the still-open critical items against the current tree.

3. Architecture overview

Confirmed from code (see `audit-reports/01-repository-and-architecture.md`):

```
Browser (React 19 client components)         Expo mobile app (Bearer JWT)
      |  lib/api.js (190 fetch wrappers)             |
      v                                              v
middleware.js: pages -> JWT role redirect; /api -> CORS only (no auth)
      |
app/api/**/route.js (180 handlers)  - requirePermission / getCurrentUserInfo / requireSuperAdmin
      |
lib/<domain>.js (students, fees, exams, attendance, ...)   lib/rbac.js, lib/iam.js, lib/auth/*
      |                                  |-- Razorpay (per-school keys, AES-GCM)  |-- Cloudflare R2
      v                                  |-- Pusher, Expo push, SMTP (console fallback)
lib/db.js (global PrismaClient) --> PostgreSQL
Server-component pages call lib/* directly. Crons (vercel.json) -> /api/cron/*
```

Stack: Next.js 16.3.2, React 19.2, JavaScript (no TypeScript), Tailwind 4, Prisma 6.19 on PostgreSQL, jose JWT (HS256, 7 days) in an httpOnly cookie or Bearer header, bcryptjs, Razorpay, Cloudflare R2, Pusher, nodemailer, Vercel region syd1. Mobile: Expo 57, React Native 0.86, Redux Toolkit, Sentry.

Trust boundaries. (1) Internet to `/api/*`: no central authentication, each handler decides. (2) Session to tenant: `schoolId` should come from the session claim but falls back to a process-global value. (3) Application to R2: public-read bucket, presigned PUT. (4) Application to Razorpay: order from database amount, HMAC verified on the callback. Items (1) and (2) are where the critical findings sit. The tenant fallback and the dual role model are inferred from code reading and from comments in `lib/auth/schoolContext.js`; they were not exercised at runtime.

Feature status (code-read, not executed): students, teachers, classes, attendance, exams and marks, fees, timetable, homework, support, calendar, parent portal and super-admin are implemented; promotion, report cards, notices, leaves, reports/exports and RBAC are partial; the online-fee webhook is missing; email delivery is partial (console fallback). Detail is in report 01, section 10.

4. Findings overview by theme

Theme	Name	Findings	Highest severity
T1	Identity fallback and global tenant singleton	6	P0
T2	Missing or wrong authorization on routes, pages and libs	7	P1
T3	Online payment and fee integrity	10	P1
T4	Identity documents and personal data exposure	8	P1
T5	Scheduled jobs (cron) are single-tenant and fail open	5	P1
T6	Vulnerable dependencies and hardening	8	P1
T7	No tests, no CI, no reproducible migrations or recovery	15	P1
T8	Performance and scalability of data loads	16	P2
T9	Weak data model: string references, JSON arrays, duplicated sources of truth	17	P1
T10	Session lifecycle, brute force and credential handling	12	P2
T11	Observability, audit trail and error handling	13	P2
T12	Business-rule gaps in exams, attendance, promotion and notifications	6	P1
T13	Frontend, accessibility and mobile polish	17	P1
T14	Maintainability and dead code	4	P3

Severity key: P0 ◆◆◆ critical, P1 ◆◆ high, P2 ◆ medium, P3 ◇ low, P4 · informational. The text label carries the meaning, so the table reads correctly in grayscale.

5. Detailed findings register

5.1 Critical and high findings (P0, P1)

Every field below comes from the auditor that owns the finding; the owning report carries the full text, reproduction notes and validation steps. "Duplicate of" lists other IDs that describe the same defect.

API-001 - Session-less requests are served as a global "dashboard Teacher" actor; unauthenticated GET returns cross-school support tickets

- **Severity:** P0 ◆◆◆ **Confidence:** High **Status:** Confirmed **Production blocker:** yes
- **Category / component:** Backend / API / lib/iam.js, lib/currentUser.js, support/exams/homework/notices routes
- **Evidence:** lib/iam.js:199-210 falls back to getDashboardRoleUser() when there is no session; lib/currentUser.js:9 keeps the role in globalThis.__CURRENT_USER__; every web login calls setCurrentRole (app/actions/auth.js:92,111,138), so the last web Teacher login makes every cookie-less request a Teacher. That fallback actor has no schoolId. Live, no cookie: GET /api/iam/me returned a Teacher user, /api/exams, /api/classes/secti…
- **Impact:** unauthenticated cross-tenant read of support tickets (subjects, messages, names, attachment URLs); unauthenticated access to every route that only checks if (!actor). State depends on which role last logged in on that server instance, so it is intermittent in serverless but permanent on a long-lived instance.
- **Recommended fix:** remove the dashboard-toggle branch from getCurrentUserInfo (return null when getSession() is empty); delete lib/currentUser.js global state or confine it to a dev flag; in getSchoolTickets and similar helpers throw if schoolId is falsy.
- **Effort:** S **Phase:** 0 **Theme:** T1 **Source:** 03-backend-and-api-audit.md
- **Validation after fix:** re-run the check described in the owning report; add a regression test where applicable.

API-002 - POST /api/schools has no authentication and repoints the global active-school singleton

- **Severity:** P1 ◆◆ **Confidence:** High **Status:** Confirmed **Production blocker:** yes
- **Category / component:** Backend / API / app/api/schools/route.js
- **Evidence:** app/api/schools/route.js:21-53 — no requireSuperAdmin (only the GET at line 7 has it). It calls addSchoolFromWizard (lib/schools.js:120, creates a School), then setActiveSchoolContext(school) (lib/school.js, mutates globalThis.__ACTIVE_SCHOOL__) and addSession/setActiveSession.
- **Impact:** anyone can create tenants (code collision probing, junk rows) and repoint the instance's fallback school, which affects the cron job, SuperAdmin sessions and API-001 actors for all later requests on that instance.
- **Recommended fix:** const { error } = await requireSuperAdmin(); if (error) return error; at the top; pass the new school id explicitly to session creation instead of mutating the singleton.
- **Effort:** S **Phase:** 0 **Theme:** T1 **Source:** 03-backend-and-api-audit.md
- **Validation after fix:** re-run the check described in the owning report; add a regression test where applicable.

API-003 - Five handlers have no auth check on any path (fee ledger, create-order, verify, attendance by student, subjects GET)

- **Severity:** P1 ◆◆ **Confidence:** High **Status:** Confirmed **Production blocker:** yes
- **Category / component:** Backend / API / see inventory
- **Evidence:** ledger route.js:5-18 only blocks a Parent for a different child, never 401s on null; create-order/route.js:11-17 and verify/route.js:11-17 run the ownership check only when actor?.role === 'Parent', otherwise proceed; attendance/student/[id]/route.js:20-22 same pattern; subjects/route.js:11. Contrast the sibling fees/student/[studentId]/route.js which was fixed.
- **Impact:** payment history and attendance of any student readable without login; unauthenticated Razorpay order creation (creates PENDING Payment rows and live Razorpay orders: abuse/spam); unauthenticated verify.
- **Recommended fix:** use requireParent/requirePermission('fees.view') or at minimum if (!actor) 401 plus ownership check for any non-admin role; add a shared requireActor() helper.
- **Effort:** S **Phase:** 0 **Theme:** T2 **Source:** 03-backend-and-api-audit.md
- **Validation after fix:** re-run the check described in the owning report; add a regression test where applicable.

API-004 - Roles/permissions are global (not per tenant); any holder of permissions.manage edits roles for all schools

- **Severity:** P1 ◆◆ **Confidence:** High **Status:** Confirmed **Production blocker:** yes
- **Category / component:** Backend / API / app/api/roles/[id]/permissions, prisma Role
- **Evidence:** prisma Role has key @unique and no schoolId (schema.prisma:70-82); app/api/roles/[id]/permissions/route.js replaces RolePermission rows of that single global role; prisma/rbacPermissions.js:98 gives PRINCIPAL = ALL_KEYS which includes permissions.manage (:70).
- **Impact:** a Principal of school A can change what Admin/Teacher/Accountant may do in every other school (including granting themselves nothing more, but revoking fees access platform-wide, or giving Teachers students.delete).
- **Recommended fix:** restrict permissions.manage to SuperAdmin, or make roles per-school.
- **Effort:** S **Phase:** 1 **Theme:** T1 **Source:** 03-backend-and-api-audit.md
- **Validation after fix:** re-run the check described in the owning report; add a regression test where applicable.

ARC-001 - Student documents (birth cert, Aadhaar, SLC) uploaded to a public-read R2 prefix with predictable keys

- **Severity:** P1 ◆◆ **Confidence:** Medium **Status:** Suspected **Production blocker:** yes
- **Category / component:** Architecture / Storage / privacy
- **Evidence:** lib/storage.js:20-26 (public URL, comment requires public-read bucket); app/api/students/document-upload-url/route.js:28 key = students/<schoolId>/documents/<Date.now()>-<safeName>; app/api/support/upload-url/route.js:33 same pattern; route.js:19-25 size/type trusted from the client.

- **Impact:** identity documents of minors/parents are readable by anyone who obtains or guesses the URL (millisecond timestamp plus a usually predictable filename such as aadhaar.pdf); the URL is persisted and returned to every client that renders the student. No access revocation or audit is possible; deleting a student does not necessarily delete the object. This is a privacy/regulatory exposure (India's DPDP Act for children'...
- **Recommended fix:** minimum: add crypto.randomUUID() to every key. Better: keep sensitive prefixes (students/*/documents/, Aadhaar) in a private bucket and serve through an authenticated route that returns a short-lived presigned GET (@aws-sdk/s3-request-presigner is already installed). Also pass ContentLength via a presigned POST policy or re-check the object size with HeadObject on confirm.
- **Effort:** M **Phase:** 1 **Theme:** T4 **Duplicate of:** PRV-002 **Source:** 01-repository-and-architecture.md
- **Validation after fix:** re-run the check described in the owning report; add a regression test where applicable.

BIZ-001 - Fee-generation cron processes one school only (global singleton), and fails open if CRON_SECRET is unset

- **Severity:** P1 ◆◆ **Confidence:** High **Status:** Confirmed **Production blocker:** yes
- **Category / component:** Backend / API / lib/fees.js:583, app/api/cron/generate-fees
- **Evidence:** lib/fees.js:583-607 generateDueStudentFees calls getFeeStructures() which resolves the school via resolveSchoolId() (lib/fees.js:217) with no session, i.e. the global singleton (lib/school.js, defaults to a seed id). app/api/cron/generate-fees/route.js:12-15 and cleanup-support-tickets/route.js only check the bearer when process.env.CRON_SECRET is set (fail-open).
- **Impact:** in a multi-school deployment only one school gets auto-generated fees; the rest silently never do. If the secret is missing the endpoints are publicly callable (mass generation / ticket purge).
- **Recommended fix:** loop prisma.school.findMany({status:'Active'}) and pass schoolId into every lib call; return 500 when CRON_SECRET is unset.
- **Effort:** M **Phase:** 0 **Theme:** T5 **Duplicate of:** OBS-003 **Source:** 03-backend-and-api-audit.md
- **Validation after fix:** re-run the check described in the owning report; add a regression test where applicable.

BIZ-002 - No Razorpay webhook; verification relies on the browser callback; abandoned checkouts lose captured payments

- **Severity:** P1 ◆◆ **Confidence:** High **Status:** Confirmed **Production blocker:** yes
- **Category / component:** Backend / API / lib/fees.js:1147-1265
- **Evidence:** no webhook route exists under app/api; lib/fees.js:1186-1193 comment acknowledges it. verifyRazorpayPayment (lib/fees.js:1194-1265) is reached only from the client POST.
- **Impact:** if the parent's browser/app closes, loses network, or the callback fails after Razorpay captures money, the Payment row stays PENDING forever (no cleanup job, comment at lib/fees.js:1142) and the fee stays unpaid: money taken, school sees no payment. No reconciliation job either.
- **Recommended fix:** add app/api/payments/razorpay/webhook/route.js verifying X-Razorpay-Signature over the raw body with the school's webhook secret, resolving the Payment by razorpayOrderId, and reusing one shared applySuccessfulPayment(paymentId) that is idempotent and runs in a transaction with a conditional update (updateMany where status:'PENDING').
- **Effort:** M **Phase:** 1 **Theme:** T3 **Duplicate of:** OBS-007, FE-001, ARC-002 **Source:** 03-backend-and-api-audit.md
- **Validation after fix:** re-run the check described in the owning report; add a regression test where applicable.

BIZ-003 - Online/manual payment races and overpayment: stale read, absolute write, many PENDING orders per fee, no cap

- **Severity:** P1 ◆◆ **Confidence:** High **Status:** Confirmed **Production blocker:** yes
- **Category / component:** Backend / API / lib/fees.js:1025-1265
- **Evidence:** collectManualPayment (lib/fees.js:1025-1081) reads the fee, computes newPaidAmount = fee.paidAmount + collectAmount in JS and writes it as an absolute value; no row lock, no idempotency key. verifyRazorpayPayment (fees.js:1239-1251) does the same with payment.amount, never re-checking that the amount is still owed. createRazorpayOrder (fees.js:1147) creates a new PENDING order on every call with no reuse or cap.
- **Impact:** (1) two concurrent manual collections both pass the <= remainingDue check, create two Payment rows, but paidAmount is overwritten with one value (ledger sum greater than paidAmount); (2) a parent who opened two checkouts, or paid online after an admin collected cash, can have both verified: paidAmount exceeds payable, status PAID, no refund path (BIZ-005); (3) double-click on collect creates duplicate Payments.
- **Recommended fix:** use atomic updates: prisma.studentFee.updateMany({ where: {id, schoolId, paidAmount: fee.paidAmount}, data:{ paidAmount: {increment: amt}}}) inside the transaction and abort on count===0; in verify re-compute remaining due and reject/flag overpay; reuse an existing PENDING order for the same fee; accept an Idempotency-Key on /api/fees/collect.
- **Effort:** M **Phase:** 1 **Theme:** T3 **Source:** 03-backend-and-api-audit.md
- **Validation after fix:** re-run the check described in the owning report; add a regression test where applicable.

BIZ-004 - Attendance lock rules are disabled in code; markedBy and records are client-controlled

- **Severity:** P1 ◆◆ **Confidence:** High **Status:** Confirmed **Production blocker:** yes
- **Category / component:** Backend / API / lib/attendance.js:103-110, app/api/attendance/route.js
- **Evidence:** lib/attendance.js:103-110 ATTENDANCE_LOCK_ENABLED = false makes computeAttendanceAccess always allow; the manual unlock (manuallyUnlocked, attendance/[id]/lock) therefore has no effect. app/api/attendance/route.js passes the raw body to saveAttendance (lib/attendance.js:57), which upserts markedBy (client supplied, DailyAttendanceBoard.jsx:202) and records (no schema check: statuses, studentIds belonging to the clas...
- **Impact:** a Teacher can alter any past or future date for their class, forge markedBy, and write arbitrary JSON into records, corrupting reports. Concurrent submissions are last-write-wins via the [school, session, date, class, section] unique upsert (no duplicate rows, which is good).
- **Recommended fix:** turn the flag on (or delete the dead matrix and decide the policy), derive markedBy from the session, validate records against the roster and ATTENDANCE_STATUSES, reject future dates.
- **Effort:** S **Phase:** 1 **Theme:** T12 **Source:** 03-backend-and-api-audit.md
- **Validation after fix:** re-run the check described in the owning report; add a regression test where applicable.

DB-001 - Migration history does not represent the schema (1 init migration, 2 of 52 models)

- **Severity:** P1 ◆◆ **Confidence:** High **Status:** Confirmed **Production blocker:** yes
- **Category / component:** Database / Migrations / DevOps

- **Evidence:** prisma/migrations/20260830172048_init/migration.sql:1-38 creates only the AdminStatus enum and the SuperAdmin and SchoolAdmin tables; prisma/schema.prisma defines 52 models. git log -- prisma/migrations shows only the initial commit. _prisma_migrations in the local DB has exactly one row. package.json:7 build script is prisma generate && next build (no prisma migrate deploy). The schema header comments and the exist…
- **Impact:** A new environment (staging, DR restore target, a second region) cannot be created from the repo; prisma migrate deploy would produce a database missing 50 tables. No reviewable, reversible record of schema change; no safe path for additive changes with data (e.g. converting date strings to DateTime). Production schema drift cannot be detected.
- **Recommended fix:** Baseline: generate a full-schema migration from the production-equivalent schema (prisma migrate diff --from-empty --to-schema-datamodel prisma/schema.prisma --script), mark it applied on existing databases with prisma migrate resolve --applied, and from then on use migrate dev/migrate deploy in the release process.
- **Effort:** M **Phase:** 1 **Theme:** T7 **Source:** 05-database-and-data-integrity-audit.md
- **Validation after fix:** re-run the check described in the owning report; add a regression test where applicable.

DB-002 - Manual fee collection is a read-modify-write outside the transaction: double-collect / overpay race

- **Severity:** P1 ◆◆ **Confidence:** High **Status:** Confirmed **Production blocker:** yes
- **Category / component:** Database / Fees / Payment
- **Evidence:** lib/fees.js:1025-1058 (collectManualPayment): reads the fee with getStudentFeeById, checks status === 'PAID' and remainingDue in JS, computes newPaidAmount = fee.paidAmount + collectAmount, then runs $transaction([payment.create, studentFee.update({ paidAmount: newPaidAmount })]). The transaction does not re-read or lock the fee, and paidAmount is written as an absolute value. lib/fees.js:1194-1250 (verifyRazorpayPa…
- **Impact:** Two accountants (or a double click, or a manual collect racing a pending Razorpay verify for the same fee) both pass the "not PAID" check; two SUCCESS Payment rows are created while paidAmount reflects only one (or a Razorpay verify adds on top of a manual full payment). Ledger and fee row then disagree; the school's reconciliation breaks and parents can be double-charged. Local DB: fee rows are currently consistent…
- **Recommended fix:** Inside one interactive transaction do the guarded update: updateMany({ where: { id, schoolId, paidAmount: fee.paidAmount }, data: { paidAmount: { increment: amount }, status } }) and abort if count === 0; or SELECT … FOR UPDATE via $queryRaw. For Razorpay mark the pending Payment SUCCESS with updateMany({ where: { id, status: 'PENDING' } }) first and only continue if it matched.
- **Effort:** S **Phase:** 1 **Theme:** T3 **Source:** 05-database-and-data-integrity-audit.md
- **Validation after fix:** re-run the check described in the owning report; add a regression test where applicable.

DB-003 - Login identity is looked up by email only, but uniqueness is per school (or absent): ambiguous account resolution

- **Severity:** P1 ◆◆ **Confidence:** High **Status:** Confirmed **Production blocker:** yes
- **Category / component:** Database / Auth identity tables
- **Evidence:** Schema: Teacher has no unique on loginEmail or email (only @@unique([schoolId, employeeId]), schema.prisma:678); ParentAccount is @@unique([schoolId, email]) (285); User is @@unique([schoolId, email]) (153); SchoolAdmin.email is globally unique (322-326). Lookups: lib/teachers.js:436-440 and :460-463 findFirst({ where: { loginEmail: … } }) with no school; lib/parentAccounts.js:108-111 findFirst by email only; lib/…
- **Impact:** Two schools can legitimately register the same parent or teacher email (the schema permits it), after which findFirst returns an arbitrary row: the person logs into the wrong tenant, or a correct password for school B fails because school A's row was returned first (a person who is a parent at two schools cannot sign in to one). Within a school, duplicate teacher login emails make password-reset and login nondetermi…
- **Recommended fix:** Decide the product rule. Simplest consistent option: login emails are globally unique across tenants (unique index on lowercased email per identity table, or a single User identity table), with a data-cleanup migration first. Alternative: require a school code at login and look up { schoolId, email }. Add @@unique([schoolId, loginEmail]) to Teacher at minimum.
- **Effort:** M **Phase:** 1 **Theme:** T9 **Duplicate of:** REL-003, SEC-013 **Source:** 05-database-and-data-integrity-audit.md
- **Validation after fix:** re-run the check described in the owning report; add a regression test where applicable.

FE-001 - Online-payment finalisation depends only on browser callback; no Razorpay webhook

- **Severity:** P1 ◆◆ **Confidence:** Medium **Status:** Confirmed **Production blocker:** yes
- **Category / component:** Frontend / fees / Razorpay (web and mobile)
- **Evidence:** components/fees/RazorpayPayButton.jsx:63-80; no webhook route under app/api
- **Impact:** Captured payments can remain unrecorded if verify call fails; parent may pay twice
- **Recommended fix:** Add signature-verified idempotent webhook; recovery message on verify failure
- **Effort:** M **Phase:** 1 **Theme:** T3 **Duplicate of:** BIZ-002, OBS-007, ARC-002 **Source:** a2.json
- **Validation after fix:** re-run the check described in the owning report; add a regression test where applicable.

MOB-001 - Demo credentials hard-coded and displayed in production login

- **Severity:** P1 ◆◆ **Confidence:** High **Status:** Confirmed **Production blocker:** yes
- **Category / component:** Mobile / src/features/auth/LoginScreen.js
- **Evidence:** LoginScreen.js:48-58,75,117-118,263 not gated by __DEV__
- **Impact:** Release builds expose seeded account credentials; access if accounts exist in production
- **Recommended fix:** Gate with __DEV__, delete/rotate seeded accounts in production
- **Effort:** S **Phase:** 1 **Theme:** T13 **Source:** a2.json
- **Validation after fix:** re-run the check described in the owning report; add a regression test where applicable.

OBS-003 - Daily fee-generation cron only processes one fallback school, not every school

- **Severity:** P1 ◆◆ **Confidence:** High **Status:** Confirmed **Production blocker:** yes
- **Category / component:** Observability / Privacy / api/cron/generate-fees, lib/fees.js
- **Evidence:** generateDueStudentFees() calls getFeeStructures() (no school argument); that resolves school through resolveSchoolId(), which with no session (a cron has none — the function's own comment at fees.js:579-583 says so) returns the process-wide getActiveSchoolId() singleton

(schoolContext.js:26, default seed id at lib/school.js:5,17).

- **Impact:** On a multi-school deployment the daily job generates term fees for at most one school (whichever the instance last "stepped into", else the seed id); all others are silently skipped while the route returns success: true. Parents of other schools are not billed on time; no one is alerted.
- **Recommended fix:** In the cron, const schools = await prisma.school.findMany({select:{id:true}}) and call a school-parameterised generator for each, collecting per-school success/failure in the response.
- **Effort:** M **Phase:** 0 **Theme:** T5 **Duplicate of:** BIZ-001 **Source:** 11-observability-and-privacy-audit.md
- **Validation after fix:** re-run the check described in the owning report; add a regression test where applicable.

OBS-007 - No Razorpay webhook or reconciliation; paid-but-unrecorded payments possible

- **Severity:** P1 ◆◆ **Confidence:** High **Status:** Confirmed **Production blocker:** yes
- **Category / component:** Observability / Privacy / Payments
- **Evidence:** only create-order and verify routes exist under app/api/payments/ (directory listing); lib/fees.js:1191,1257 comments say a webhook is a future addition ("once built"). Payment rows start PENDING (schema.prisma:1437) and become SUCCESS only when the browser/app calls verify (fees.js:1236-1248). No job re-queries Razorpay for stale PENDING rows, and no admin screen lists them. Also create-order/route.js:13 and verify...
- **Impact:** If a parent pays and the app/tab closes, the network drops, or the callback fails, money is captured at Razorpay but the fee stays unpaid and the Payment stays PENDING indefinitely; staff must reconcile manually against the Razorpay dashboard. Refunds do not exist either (prior audit P28). The null-actor path lets an unauthenticated caller create orders/PENDING rows against any fee id (signature check still protects...
- **Recommended fix:** Add POST /api/payments/razorpay/webhook (verify X-Razorpay-Signature with the webhook secret over the raw body; call the existing idempotent confirm path which already tolerates duplicates, fees.js:1256-1262), plus a daily/admin "stale PENDING payments" check that fetches the order status. Require a signed-in actor on create-order/verify.
- **Effort:** M **Phase:** 1 **Theme:** T3 **Duplicate of:** BIZ-002, FE-001, ARC-002 **Source:** 11-observability-and-privacy-audit.md
- **Validation after fix:** re-run the check described in the owning report; add a regression test where applicable.

OPS-001 - No CI/CD gates in either repo

- **Severity:** P1 ◆◆ **Confidence:** High **Status:** Confirmed **Production blocker:** yes
- **Category / component:** DevOps / both repos
- **Evidence:** no .github directory in ADMIN or MOBILE; package.json:5-11
- **Impact:** Broken or insecure code deploys unchecked; about 100 uncommitted modified files obscure release provenance.
- **Recommended fix:** GitHub Actions: npm ci, lint, prisma validate, next build with dummy env, tests; protect main.
- **Effort:** S **Phase:** 1 **Theme:** T7 **Source:** a1.json
- **Validation after fix:** re-run the check described in the owning report; add a regression test where applicable.

PRV-001 - Parent and Accountant roles can read any student's full record (Aadhaar, guardians, address)

- **Severity:** P1 ◆◆ **Confidence:** High **Status:** Confirmed **Production blocker:** yes
- **Category / component:** Observability / Privacy / GET /api/students/[id]
- **Evidence:** app/api/students/[id]/route.js:7-19 — gate is requirePermission('students.view') followed by a block that rejects only user.roleKey === 'Teacher'; the comment (lines 10-16) states the route is meant to be "admin-tier only". prisma/rbacPermissions.js grants students.view to Parent (line ~205) and Accountant (line 195) as well as Teacher/Admin. lib/students.js:310-312 returns the full decorated record by {id, schoolId...
- **Impact:** A signed-in parent can fetch any other student in the same school (ids are cuids, but they are exposed in rosters, notices, links) and read minors' Aadhaar numbers, guardian contacts and addresses. Accountant (no teaching/admin need) gets the same. Same-school only; no cross-tenant leak.
- **Recommended fix:** In the route, allow only roles that legitimately need the full record: if (!['Admin','Principal','SuperAdmin'].includes(user.roleKey)) return 403; Parent already has dedicated parent routes. Consider returning a reduced projection for Accountant.
- **Effort:** S **Phase:** 1 **Theme:** T2 **Source:** 11-observability-and-privacy-audit.md
- **Validation after fix:** re-run the check described in the owning report; add a regression test where applicable.

PRV-002 - Aadhaar and identity documents stored under permanent unsigned public URLs

- **Severity:** P1 ◆◆ **Confidence:** Medium **Status:** Suspected **Production blocker:** yes
- **Category / component:** Observability / Privacy / R2 storage / document upload
- **Evidence:** lib/storage.js:20-27 — buildPublicUrl returns ${R2_PUBLIC_URL}/${key} and its comment states the bucket must allow public reads; there is no signed-GET path in the file. Student document keys are students/<schoolId>/documents/<Date.now()>-<sanitised original filename> (students/document-upload-url/route.js:26-28); teacher equivalent teachers/document-upload-url/route.js:22-24. The URL is stored in the DB (Student.aa...
- **Impact:** Anyone holding or guessing a URL can download a child's or teacher's Aadhaar/birth-certificate scan with no authentication, no expiry and no access log. Keys are not high-entropy (millisecond timestamp plus a user-chosen filename such as the document type), and URLs leak through any log, referrer, shared screenshot or the PRV-001 route.
- **Recommended fix:** Smallest change: keep photos public, but store identity documents under a separate private bucket (no public domain) with a random key (crypto.randomUUID()), and serve them via a permission-checked route that returns a short-lived presigned GET (GetObjectCommand + getSignedUrl, 60-300 s). Existing objects need migration.
- **Effort:** M **Phase:** 1 **Theme:** T4 **Duplicate of:** ARC-001 **Source:** 11-observability-and-privacy-audit.md
- **Validation after fix:** re-run the check described in the owning report; add a regression test where applicable.

SEC-001 - Any signed-in Parent (and Accountant) can write homework and exam marks: routes only require a non-null actor and libs treat every non-Teacher as admin

- **Severity:** P1 ◆◆ **Confidence:** High **Status:** Confirmed **Production blocker:** yes
- **Category / component:** Authorization / homework + exam marks APIs

- **Evidence:** lib/examMarks.js:32-33,164-169,243-247; lib/homework.js:201-203,231-232,269-281; app/api/homework/route.js:21-47; app/api/exams/schedules/[scheduleId]/marks/route.js:33-52; app/api/homework/[id]/route.js:5-28 (code-based, not executed)
- **Impact:** A parent token (mobile login) can POST/PUT/DELETE homework for any class (with push fan-out to students), read a full class marks sheet and upsert marks for any studentId. Academic-record integrity and spam of every parent in the school.
- **Recommended fix:** Replace the "role !== Teacher => allow" shortcut with explicit allow-lists (SchoolAdmin/SuperAdmin) and add requirePermission (marks.enter/marks.update, homework manage) at the route; add a negative test per role.
- **Effort:** S **Phase:** 1 **Theme:** T2 **Source:** a4.json
- **Validation after fix:** re-run the check described in the owning report; add a regression test where applicable.

SEC-002 - Dashboard server pages have no server-side role/permission guard: a signed-in Teacher can render Teachers, Fees, Settings and Promote pages with SSR data

- **Severity:** P1 ◆◆ **Confidence:** Medium **Status:** Suspected **Production blocker:** yes
- **Category / component:** Authorization / dashboard pages (teachers, fees, settings, promote)
- **Evidence:** app/dashboard/teachers/[id]/page.jsx:15-18; app/dashboard/teachers/page.jsx:14-18; app/dashboard/fees/payments/page.jsx:9; app/dashboard/fees/students/page.jsx:15-31; app/dashboard/layout.jsx:10-17; middleware.js:26-30,86-94 (middleware only checks role tier). Not executed.
- **Impact:** Hidden sidebar links are the only barrier: typing the URL returns data fetched directly from lib (bypassing API requirePermission), including teacher HR/financial fields (PRV-003) and school-wide fee payments.
- **Recommended fix:** Add a shared page guard (requirePermission-equivalent for pages) to every dashboard page, or one in app/dashboard/layout per route segment; verify live per role.
- **Effort:** M **Phase:** 1 **Theme:** T2 **Source:** a4.json
- **Validation after fix:** re-run the check described in the owning report; add a regression test where applicable.

SEC-004 - Process-global tenant and role singletons are mutated by every school-admin/teacher login, so SuperAdmin and session-less requests act on whichever school logg...

- **Severity:** P1 ◆◆ **Confidence:** High **Status:** Confirmed **Production blocker:** yes
- **Category / component:** Tenant isolation / lib/school.js, lib/currentUser.js, auth actions
- **Evidence:** app/actions/auth.js:76-77,92,100-101,111,138; lib/school.js:29-31,43-45; lib/auth/schoolContext.js:23-27; app/api/schools/[id]/manage/route.js:21-24
- **Impact:** SuperAdmin sessions carry no schoolId, so resolveSchoolId() returns the global singleton: a SuperAdmin who stepped into school A can have reads and writes land in school B after a school-B admin logs in. Cross-tenant misdirected writes and disclosure.
- **Recommended fix:** Remove active-school and dashboard-role singletons; carry the managed schoolId inside the SuperAdmin token or a per-request header validated server-side; resolveSchoolId must throw when no schoolId.
- **Effort:** L **Phase:** 0 **Theme:** T1 **Source:** a4.json
- **Validation after fix:** re-run the check described in the owning report; add a regression test where applicable.

SEC-008 - next 16.3.2 is in the affected range of critical and high advisories (image optimisation RCE/SSRF, next/og RCE, cache poisoning)

- **Severity:** P1 ◆◆ **Confidence:** Medium **Status:** Confirmed **Production blocker:** yes
- **Category / component:** Dependencies / package.json:24
- **Evidence:** package.json:24 (next 16.3.2 pinned); npm audit --json: next critical, fix 16.4.1 (non-major). App imports next/image in components/LoginForm.jsx and uses no next/og; exploitability depends on image config and hosting (not verified).
- **Impact:** Known unauthenticated RCE/SSRF classes in the framework; presence confirmed, exploitability not verified.
- **Recommended fix:** Upgrade next to the patched release (16.4.1 or later), re-run npm audit, and review images config.
- **Effort:** S **Phase:** 0 **Theme:** T6 **Source:** a4.json
- **Validation after fix:** re-run the check described in the owning report; add a regression test where applicable.

SEC-017 - API-001 fallback actor also reaches write paths: support ticket replies/reads are unscoped when actor.schoolId is undefined; marks and homework run as the togg...

- **Severity:** P1 ◆◆ **Confidence:** High **Status:** Confirmed **Production blocker:** yes
- **Category / component:** Authorization / lib/iam.js, lib/support.js, support API
- **Evidence:** lib/iam.js:197-210; lib/support.js:63-65,110-112,155-158 (undefined schoolId drops the where filter in Prisma); app/api/support/tickets/[id]/replies/route.js:5-19; app/api/support/tickets/[id]/route.js:13 (code-based; dev server not running so not re-tested live)
- **Impact:** Beyond the read leak in API-001: a session-less caller can read any ticket by id, list all tenants' tickets and post replies into any tenant's thread (triggering notifications), when a Teacher web login previously ran on that instance.
- **Recommended fix:** Fix API-001 at the root; make support helpers throw on falsy schoolId.
- **Effort:** S **Phase:** 0 **Theme:** T1 **Source:** a4.json
- **Validation after fix:** re-run the check described in the owning report; add a regression test where applicable.

TST-001 - No automated tests in ADMIN (no runner, no test script)

- **Severity:** P1 ◆◆ **Confidence:** High **Status:** Confirmed **Production blocker:** no
- **Category / component:** Other / whole repo
- **Evidence:** package.json:5-11 no test script; package.json:13-45 no test framework
- **Impact:** Auth, tenant isolation, fees and payments have no regression net across ~180 routes.
- **Recommended fix:** Add node --test or vitest; start with fee calc, JWT/permission helpers, signature check, two-school isolation fixture.
- **Effort:** L **Phase:** 1 **Theme:** T7 **Source:** a1.json
- **Validation after fix:** re-run the check described in the owning report; add a regression test where applicable.

5.2 Medium, low and informational findings (P2, P3, P4)

ID	Severity	Conf.	Title	Component / evidence	Fix	Effort	Phase
API-005	P2 ◆	High	No server-side schema validation; client-only zod; malformed JSON gives 500	all POST/PUT routes	a 10-line parseBody(request, schema) helper using the existing zod schemas (reuse feeStructureSchema, manualPaymentSche…	M	1
API-006	P2 ◆	High	Unbounded list responses and unbounded request arrays	students, users, notices, bulk-collect, bulk-import	cap array lengths (e.g. 500) and pageSize; migrate remaining list consumers to the paged routes.	S	2
API-007	P2 ◆	Med	Upload presign does not bind size; free-form attachmentUrl/photoUrl stored; cleanup cron can delete another t…	upload-url routes, lib/support.js:229	validate attachmentUrl/photoUrl start with R2_PUBLIC_URL + '/<prefix>/<schoolId>/'; use a presigned POST with content-l…	S	1
API-008	P2 ◆	High	Rate limiting exists only on logins and is in-memory per instance	lib/auth/loginRateLimit.js	a shared store (Upstash/Postgres) for login; per-user caps on order creation and ticket creation.	M	1
ARC-002	P2 ◆	High	No Razorpay webhook: payment recording depends on the browser returning to the verify endpoint	Fees / payments	add app/api/payments/razorpay/webhook/route.js (raw body, X-Razorpay-Signature HMAC with a per-school webhook secret, h…	M	1
BIZ-005	P2 ◆	High	No refund/reversal path; no collector or actor recorded on payments and discounts	fees module	add Payment.collectedById/collectedByName, a FeeAuditLog model written in the same transaction, and a POST /api/fees/pa…	M	2
BIZ-006	P2 ◆	High	Marks edits after approval/publish do not invalidate or recompute results; marks save is non-transactional	lib/examMarks.js:164-258, lib/examResults.js	when a mark of a published exam changes, flag the student's ExamResult stale (or recompute) and keep status; wrap the r…	M	2
BIZ-007	P2 ◆	High	Attendance reports key history on the student's CURRENT class/section	lib/attendance.js:190-196	join against StudentEnrollment for the date range or scan all records blobs for the studentId.	M	2
BIZ-008	P2 ◆	Med	Promotion: promote and retain run as parallel transactions; no status/target validation; no fee/dues handling	app/api/students/promote, lib/students.js:738-875	run the two operations sequentially in one transaction, filter status: 'Active', call generateFeesForNewStudent after p…	M	2
BIZ-009	P2 ◆	Med	Notification/push fan-out is fire-and-forget (un-awaited) on serverless; no retry or outbox	lib/notices.js:542, lib/fees.js:1052	use after()/waitUntil (see node_modules/next/dist/docs for the current API) or await inside the request.	S	2
DB-004	P2 ◆	High	Missing indexes: no schoolId-leading index on Payment, StudentFee, Notice, CalendarEvent; unindexed FK columns	Indexes	@@index([schoolId, status, paidAt]) and @@index([studentId]), @@index([studentFeeId]) on Payment; @@index([schoolId, ac…	S	2
DB-005	P2 ◆	High	Hard delete of a Student cascades away exam results, marks, enrollment history and the "append-only" promotio…	Deletion / archival	Make delete = status change (or an archivedAt DateTime? column) for any student with enrollments/results; restrict hard…	M	2
DB-006	P2 ◆	High	All business dates are String; school timezone setting is never used to compute "today"	Date design	Smallest effective step: one schoolToday(timezone) helper (using Intl.DateTimeFormat('en-CA', { timeZone })) used by ev…	S	2
DB-007	P2 ◆	High	Attendance / assignments / timetable stored as JSON arrays of ids with no FK, no per-student uniqueness, last…	Attendance, Teacher.assignments	Keep the row-per-section shape but (1) add an updatedAt/version precondition to saveAttendance, (2) validate that every…	M	2
DB-008	P2 ◆	Med	Seed script is stale against the schema and embeds plaintext demo credentials and fixed school-1 ids	prisma/seed.js	Split into seed-reference.js (roles, permissions, grade scale: safe for prod) and seed-demo.js (guarded by NODE_ENV !==…	S	2
FE-002	P2 ◆	High	No error.jsx or global-error.jsx in app tree	app routing	Add error.jsx for dashboard, parent, super-admin and global-error.jsx	S	2
FE-003	P2 ◆	High	ESLint: 55 errors incl. 47 set-state-in-effect	components (47 sites)	Key-remount modal forms; fix purity errors	M	2
FE-004	P2 ◆	Med	No global 401/expired-session handling in client fetch layer	lib/api.js	Shared apiFetch wrapper redirecting on 401	M	2
MOB-002	P2 ◆	High	Real mobile app not under version control; config files untracked	schoolapp360 repo	Commit app, add private remote, keep adminsdk key and keystore out	S	1
MOB-003	P2 ◆	High	No 401 handling; logout does not reset RTK Query cache	src/api/baseApi.js, ProfileScreen.js	401 -> clear and logout; resetApiState on logout	S	2
MOB-004	P2 ◆	High	Push token never unregistered on logout	usePushNotifications, lib/pushTokens.js	Add unregister endpoint and call on logout	S	2
MOB-005	P2 ◆	Med	Razorpay verify failure shown as Payment failed after capture	ParentFeesScreen.js	Separate verify failure message, refetch fees, add webhook	S	2
OBS-001	P2 ◆	High	No server-side error tracking, request IDs or structured logs on the web app	Observability	@sentry/nextjs with instrumentation.js + sendDefaultPii:false and a beforeSend scrubber (see PRV-007); add a tiny lib/l…	M	2
OBS-004	P2 ◆	High	Cron endpoints fail open if CRON_SECRET is unset	api/cron/*	if (!process.env.CRON_SECRET // authHeader !== \Bearer ${process.env.CRON_SECRET}\) return 401.	S	0
OBS-005	P2 ◆	High	Cron/job failures are swallowed: HTTP 200 and no alert	generateDueStudentFees	push {structureId, term, error: err.message} into a failures array, return HTTP 500 when non-empty, and notify via an a…	S	2
OBS-006	P2 ◆	High	Push / realtime / in-app notification failures are console-only, no retry or record	Notifications	minimum: count failures per run and report them through the Sentry/log helper from OBS-001 with an alert threshold; opt…	S	2
OBS-008	P2 ◆	High	Most sensitive actions have no audit trail	Fees, roles, deletes, settings	one generic AuditLog model (schoolId, actorId, actorRole, action, entityType, entityId, meta Json, createdAt) and a 10-…	M	2

ID	Severity	Conf.	Title	Component / evidence	Fix	Effort	Phase
OBS-010	P2 ◆	Med	No backup/restore or incident-response documentation	Operations	one-page runbook: backup tier and retention, last restore test date, who to call, steps to rotate JWT/R2/Razorpay secre…	S	2
OPS-002	P2 ◆	High	Cron endpoints fail open when CRON_SECRET is unset	app/api/cron	Reject when the secret is missing: if (!s // header !== 'Bearer ' + s) return 401.	S	0
OPS-003	P2 ◆	High	No env validation, no ADMIN .env.example, localhost URL fallback	configuration	Startup env check in instrumentation.js and a names-only .env.example.	S	1
OPS-004	P2 ◆	High	No error tracking, structured logging or health endpoint in ADMIN	observability	Add /api/health (SELECT 1) and Sentry via instrumentation.js.	M	2
OPS-005	P2 ◆	Med	Migrations manual with single baseline; no backup/DR evidence	database operations	Explicit migrate deploy release step, confirm provider PITR, restore runbook.	M	1
OPS-006	P2 ◆	Med	Login rate limit stored in process memory	lib/auth	Persist attempts in Postgres keyed by ip+email.	M	2
OPS-009	P2 ◆	Med	Mobile release assets (keystore, eas.json, firebase files) exist only locally	schoolapp360 release	Private remote for source; keystore in EAS plus offline copy; move Firebase admin key out of project dir.	S	1
PERF-001	P2 ◆	High	Class/section pages and student list load the entire school roster including JSON blobs	Classes and Sections pages, GET /api/students	groupBy class/section counts for list views; filtered select findMany for section detail; keep getAllStudents for expor…	M	2
PERF-002	P2 ◆	High	Fee generation loads whole roster and issues one create per student in one array transaction	generateStudentFees and daily cron	Query with where filter and narrow select, chunk, createMany skipDuplicates, export maxDuration on cron	M	2
PERF-003	P2 ◆	High	Dashboard attendance trend scans every attendance row ever stored for the school	Admin dashboard attendance chart	Bound default to current session or 12 months, or maintain a daily summary table, or cache with tag invalidation on save	M	2
PERF-004	P2 ◆	Med	No connection-pool configuration in code or schema; fan-out patterns can exhaust a serverless pool	lib/db.js, prisma/schema.prisma, Vercel	Use provider transaction pooler with connection_limit, add directUrl for migrations, bound Promise.all concurrency, doc…	S	2
PERF-005	P2 ◆	High	Bulk student import: full roster read, sequential inserts, unbounded parallel fee generation, no row cap	POST /api/students/bulk-import	Cap rows per request, duplicate check via IN query, bounded concurrency or createMany for fees	M	2
PERF-006	P2 ◆	Med	Promotion/retention does 3 sequential queries per student inside a default-timeout interactive transaction	promoteStudents, retainStudentsInSession	createMany for enrollments and logs after one lookup, explicit timeout, UI chunking	M	2
PERF-007	P2 ◆	High	Whole-school aggregates computed in JavaScript (fees dashboard, exam dashboard)	Admin dashboard fee cards, exams dashboard	groupBy/count for exams; push stable sums to SQL and compute late fee only for overdue unpaid fees in active session	M	3
PRV-003	P2 ◆	High	Aadhaar, PAN and bank details stored and served in plaintext	Student/Teacher models	(1) Return a list projection without Aadhaar/PAN/bank from paged; fetch sensitive fields only on a detail view. (2) Reu…	M	1
PRV-004	P2 ◆	High	No erasure/retention path for student/teacher personal data and documents	Delete flows	On delete/transfer-out purge document objects (documents.*Url, aadhaarDocumentUrl, guardian document URLs) via deleteOb…	M	2
PRV-005	P2 ◆	High	No privacy notice, terms or consent capture anywhere (web and mobile)	Whole product	After legal advice: publish privacy notice/terms pages, link from login screens and app stores, add a recorded acceptan…	M	2
REL-001	P2 ◆	High	Fee and payment relationship: denormalised paidAmount with unconstrained Payment links	StudentFee, Payment	Guarded atomic increment, CHECK constraints, FK indexes, reconciliation job	M	3
REL-002	P2 ◆	High	Student delete cascade policy is inconsistent across child tables	Student and children	Archive instead of delete; Restrict on results and promotion log (see DB-005)	M	3
REL-003	P2 ◆	High	Login identities are not unique across tenants (Teacher has no unique email)	Teacher, ParentAccount, User, SchoolAdmin	Resolve per DB-003	M	3
REL-004	P2 ◆	High	Class, section and subject are plain strings in most tables; names drift from master data	Student, Attendance, Homework, ExamSchedule, TimeTable, Teacher.assignments	Block rename/delete with dependants and validate on write; longer term add nullable id FKs and backfill	L	3
SEC-003	P2 ◆	High	No session revocation: deactivated users, disabled teachers, deactivated admins and suspended schools keep ac…	session lifecycle	Re-check account and school status per request (cache briefly) or add a token version/jti stored per user; reject inact…	M	1
SEC-005	P2 ◆	High	PUT /api/users/[id] has no role-hierarchy check: an Admin can deactivate or re-email a Principal; status is n…	users API	Apply canAssignRole-style hierarchy to the target user, validate status against an enum, and block self-status change.	S	1
SEC-006	P2 ◆	High	Parent role holds permissions that expose school-wide data: class attendance rosters, defaulter ids, all-clas…	attendance roster, fees defaulters, print-marksheet	Add Parent blocks or ownership scoping to these routes; remove attendance.student.view from the Parent default set if u…	S	1
SEC-007	P2 ◆	High	No security headers (CSP, frame-ancestors, HSTS, nosniff) and CORS * on every /api route	next.config.mjs, middleware.js	Add headers() in next.config (CSP, X-Frame-Options/frame-ancestors, nosniff, Referrer-Policy, HSTS); restrict CORS to k…	S	1
SEC-014	P2 ◆	Med	Plaintext demo credentials remain in source: comment in lib/auth.js and seed admin password literals; prior c…	lib/auth.js, prisma/seed.js	Delete the comment; verify and rotate any seeded account in production; make the seed generate random passwords.	S	1
TST-002	P2 ◆	High	MOBILE jest configured but zero app tests; jest/babel-jest version mismatch	schoolapp360 testing	Align babel-jest to 29, add roots to src, add authSlice/baseApi/utils tests.	M	2

ID	Severity	Conf.	Title	Component / evidence	Fix	Effort	Phase
TST-003	P2 ◆	High	ESLint baseline red: 97 errors, 35 warnings	components	Fix unescaped entities mechanically, triage set-state-in-effect, then enforce in CI.	M	2
UX-001	P2 ◆	High	Modal lacks dialog semantics, focus trap, accessible close; backdrop discards forms	components/Modal.jsx	role=dialog, aria-modal, labelled close, focus management, optional no-backdrop-dismiss	S	2
API-009	P3 ◇	High	err.message (incl. raw Prisma/SDK text) returned to clients in ~97 places	all routes	a toApiError(err) mapper: known business errors (custom class) to 400, everything else to 500 with a generic message an…	M	3
API-010	P3 ◇	High	Temp passwords generated with Math.random and returned in the response	app/api/users/route.js:17, lib/admins.js:6, lib/parentAccounts.js:6	crypto.randomBytes(9).toString('base64url'); force change at first login.	S	3
API-011	P3 ◇	Med	Sequential awaits / N+1 / full-school loads in fee generation, bulk import, teacher student list	lib/fees.js:437, lib/students.js:540	query students with where: {class, academicSession, status}, use createMany({skipDuplicates:true}), bound concurrency.	M	3
ARC-003	P3 ◇	High	middleware.js uses the file convention deprecated in Next 16 (renamed proxy)	Framework	npx @next/codemod@canary middleware-to-proxy . (docs: version-16.md codemods) when convenient; rename function to proxy.	S	3
ARC-004	P3 ◇	High	No error.jsx / global-error.jsx anywhere in app/	Frontend resilience	add app/error.jsx and app/global-error.jsx (client components with a retry button) and ship the error to the logging so…	S	3
ARC-005	P3 ◇	Med	GeneratedReportJob model (and 3 seed-data libs, 8 components) are dead	Schema / repo	delete after a human confirms; drop the model in a real migration (see OPS-003).	S	3
BIZ-010	P3 ◇	Med	Exam status has no transition rules; re-generating results silently unpublishes	lib/exams.js:279, lib/examResults.js:207	explicit transition table.	S	3
DB-009	P3 ◇	High	Status/type fields mix real enums with free strings	Schema	Convert the closed sets (AcademicSession.status, ExamResult.status, Notice.audience) to enums when next touching them; …	S	3
DB-010	P3 ◇	High	Money design: whole-rupee Int, derived payable amount, no DB invariants, hard-coded INR/paise	Fees	Add CHECK constraints (paidAmount >= 0, amount > 0); add a per-school receipt number on SUCCESS payments; freeze late f…	M	3
DB-011	P3 ◇	High	DB-level invariants documented as "enforced in code" are not enforced (one active session, one class-teacher …	Sessions / Sections	See report	S	3
DB-012	P3 ◇	High	Child tables without schoolId (ReportLock cannot be tenant-scoped at all)	Tenant ownership	Add schoolId + index to ReportLock; for the ParentStudentLink invariant use composite FKs (schoolId, studentId) -> Stud…	S	3
FE-005	P3 ◇	High	Duplicate uncached server fetch in generateMetadata and page	sections/[sectionId], exams/[id]/verify pages	Wrap getters in React cache()	S	3
FE-006	P3 ◇	Med	Possible double fetch on filter change; heavy router.refresh use	useUrlSync, usePagedResult	Single data source per explorer	M	3
FE-007	P3 ◇	Med	No request cancellation (AbortController in 0 files)	components	Optional signal in api helpers	S	3
FE-009	P3 ◇	Med	Very large components	TimeTableClient (1023), RolesPermissionsExplorer (739), StudentForm (737)	Split when touched	L	3
MNT-001	P3 ◇	High	Oversized modules: lib/api.js 2181 lines, lib/fees.js 1728	lib	Split per domain with re-exports; extract pure fee calculators first.	M	3
MNT-002	P3 ◇	High	README is boilerplate; no env/deploy/runbook docs	docs	Rewrite README: setup, env names, deploy, migrations, crons.	S	3
MNT-003	P3 ◇	Med	xlsx installed from vendor CDN tarball	bulk import	Keep; cache tarball in CI; keep upload size caps.	S	3
MNT-004	P3 ◇	Med	Process-local globalThis/Map state used as stores	lib	Keep globalThis only for Prisma client; move other state to DB.	M	3
MOB-006	P3 ◇	High	Silent localhost API fallback; login error reveals API URL	baseApi.js:3, LoginScreen.js	Fail loudly if unset outside __DEV__	S	3
MOB-007	P3 ◇	High	No tests despite jest setup	schoolapp360 testing	Add tests for baseQuery and fee helpers	M	3
MOB-008	P3 ◇	High	No accessibility labels in mobile UI	src UI	Label shared Button and icon buttons first	M	3
MOB-009	P3 ◇	Med	Offline handling is banner-only; no draft/write queue	OfflineBanner, attendance/marks screens	Keep drafts until save succeeds	M	3
MOB-012	P3 ◇	Med	Very large mobile screen files	ParentCalendarScreen (1061), EnterMarksScreen (1039), DailyAttendanceScreen (1008)	Split when touched	L	3
OBS-002	P3 ◇	High	No error.jsx / global-error.jsx boundaries	App shell	add app/global-error.jsx and app/error.jsx that call the error reporter and show a friendly retry.	S	3
OBS-009	P3 ◇	High	No health/uptime endpoint or DB metrics	Operations	app/api/health/route.js doing prisma.$queryRaw\SELECT 1\ returning 200/503 (no data, no auth); point an uptime monitor …	S	3
OBS-011	P3 ◇	High	Dev-email fallback prints recipient and full message body to logs	lib/email.js	guard with NODE_ENV !== 'production'; in production throw so the caller reports failure.	S	3
OBS-012	P3 ◇	High	dev.log is tracked in git	Repo hygiene	git rm --cached dev.log, add *.log to .gitignore (owner action; not done here).	S	3

ID	Severity	Conf.	Title	Component / evidence	Fix	Effort	Phase
OBS-013	P3 ◇	Med	Login abuse limiter is per-instance memory with no alerting or failed-login record	Auth abuse	log a structured event on every lockout (Sentry/helper); if the abuse is observed, back the counters with a small Postg…	S	3
OPS-007	P3 ◇	High	No security headers; empty next.config	next.config.mjs	Add headers() with frame-ancestors, nosniff, Referrer-Policy.	S	3
OPS-008	P3 ◇	High	dev.log, PDFs and scratch scripts tracked in git	repo hygiene	Untrack dev.log and add *.log to .gitignore (owner decision).	S	3
PERF-008	P3 ◇	Med	Every protected request re-loads user + role + all permissions uncached; polling multiplies it	requirePermission, getCurrentUserInfo	React cache() around actor loaders; cache role permission sets with tag invalidation	S	3
PERF-009	P3 ◇	Med	Attendance reports pull JSON records for the whole range and scan in JS	Attendance reports	Narrow select, bound ranges, consider row-per-student table after benchmarking	M	3
PERF-010	P3 ◇	Med	Heavy libraries statically imported into client components; no next/dynamic anywhere	Dashboard chart modals, bulk import modals, address forms	dynamic import with ssr:false for chart and import modals; lazy-load country-state-city	S	3
PERF-011	P3 ◇	High	pageSize uncapped on payments and schools list routes; remaining unbounded list endpoints	GET /api/payments, GET /api/schools	Apply the clamp used at app/api/homework/page/route.js:24; clamp page >= 1	S	3
PERF-012	P3 ◇	Med	Notice/exam notification fan-out is O(recipients) in memory on the request process and unawaited	Notices, parent notifications	Use after()/waitUntil per installed Next docs or await bounded counts; chunk createMany; bound concurrency	M	3
PRV-006	P3 ◇	Med	Third-party processors and hosting region need a documented basis	Processors	maintain a short processor/region table in the privacy notice and DPA; confirm R2 jurisdiction and Supabase region; con…	S	3
PRV-007	P3 ◇	Med	Unredacted error objects logged; token logged in push errors	Logging	log err.code/err.message via the helper from OBS-001, drop the token from line 53-55 (log batch index only).	S	3
REL-005	P3 ◇	Med	Student placement has two sources of truth (strings on Student and StudentEnrollment FKs)	Student, StudentEnrollment	Single write function for placement; periodic consistency query	M	3
REL-006	P3 ◇	Med	No tenant off-boarding path: every School FK is Restrict, children mixed	School	Document and build an ordered offboarding job; keep Restrict	L	3
REL-007	P3 ◇	High	Duplicated helpers and business rules across lib and components	lib, components	Single formatters module and dateUtils import; one scope function used by client filters	S	3
REL-008	P3 ◇	Med	Module structure: god modules and cycle avoided only by file-splitting and comments	lib/api.js, lib/students.js, lib/fees.js	Add server-only to Prisma/next-headers modules (check installed Next docs), split lib/api.js by domain	M	3
SEC-009	P3 ◇	High	nodemailer 10.0.1 has multiple advisories (DoS, cross-transport TLS servername reuse); sharp has libheif/libr…	package.json:25; lib/email.js	Upgrade nodemailer and sharp to fixed versions.	S	3
SEC-010	P3 ◇	High	Set-password link: no server-side password policy, 7-day plaintext token, no redemption throttling, no status…	lib/teachers.js, prisma Teacher	Store a hash of the token, shorten TTL to 1-2 hours, enforce min length and common-password checks server-side, rate-li…	S	3
SEC-011	P3 ◇	High	Login rate limiter is bypassable and enables targeted lockout: client-controlled x-forwarded-for first hop, p…	lib/auth/loginRateLimit.js	Shared store (Upstash/Postgres), use platform-provided client IP, add progressive delay rather than hard lockout.	M	3
SEC-012	P3 ◇	Med	Cross-tenant reference gaps: section update not tied to class, classTeacherId and marks studentId not validat…	lib/classes.js, lib/examMarks.js	Scope every update by schoolId in the where clause (updateMany with schoolId) and validate referenced ids belong to the…	S	3
SEC-013	P3 ◇	Med	Login lookups by email are global while uniqueness is per school or absent, so same-email accounts collide	login lookups	Make login emails globally unique or require a school code at login.	M	3
SEC-016	P3 ◇	Med	Push token registration transfers ownership of an existing token to the caller	lib/pushTokens.js, register-token	Refuse re-binding of a token owned by another user unless proven by device; delete token on logout.	S	3
TST-004	P3 ◇	High	DB-mutating ad-hoc scripts tracked at repo root	repo root	Move to scripts/dev-only with a localhost-host guard, or delete.	S	3
UX-002	P3 ◇	High	Dropdown/DropdownMenu without keyboard or ARIA support	components/Dropdown.jsx, DropdownMenu.jsx	Add key handling and ARIA or use native select	M	3
UX-003	P3 ◇	High	Mixed loaders: PageLoader spinner vs shimmer skeletons	11 route loading.jsx files	Convert to skeletons progressively	M	3
UX-004	P3 ◇	Med	Unsaved-changes guard only in timetable editor	marks entry, attendance, forms	useBeforeUnload(isDirty) hook	S	3
UX-005	P3 ◇	Med	Inconsistent pagination pattern (fixed bar copies vs inline)	list pages	Single shared bar	S	3
DB-013	P4 ·	High	Dead / stale schema elements	Schema hygiene	Remove or mark as planned; refresh header comments.	S	4
FE-008	P4 ·	High	Notification link passed to router.push unvalidated	NotificationBell, ParentTopbar	Require leading single slash	S	4
MNT-005	P4 ·	High	Convention notes: deprecated middleware file, raw img tags	framework conventions	See ARC-003, ARC-004.	S	4

ID	Severity	Conf.	Title	Component / evidence	Fix	Effort	Phase
MOB-010	P4 ·	High	Client-side role selection; backend enforces from JWT (informational)	AppNavigator.js:282-285, App.js:46-56	Keep authorization server-side	S	4
MOB-011	P4 ·	Low	Generated manifest requests RECORD_AUDIO and SYSTEM_ALERT_WINDOW	android/app/src/main/AndroidManifest.xml:4-5	blockedPermissions in app.json; inspect release manifest	S	4
PERF-013	P4 ·	High	Support unread count loads every ticket row; calendar tag invalidates all tenants	Support tickets, calendar cache	count with where; per-school cache tags	S	4
PRV-008	P4 ·	Med	Realtime channels are public with predictable names (ID-only payloads)	Pusher	keep payloads ID-only (as now) or move to private- channels with an auth route if content is ever added.	S	4
SEC-015	P4 ·	High	Thirteen high-severity transitive advisories in dev tooling (eslint-config-next chain, prisma CLI chain, brac…	package-lock	Update with npm audit fix in a branch when convenient.	S	4
SEC-018	P4 ·	Low	Forgot-password timing differs for valid teacher emails (mail send awaited only on match)	lib/teachers.js	Send the mail without awaiting, or pad response time.	S	4
SEC-019	P4 ·	Med	Hardening gaps: no CSRF token (relies on SameSite=Lax, request.json ignores content type), deprecated middlew…	session/middleware	Rename to proxy.js, add iss/aud and separate secrets for cookie and bearer tokens, enforce JSON content type.	S	4

The complete register with evidence and impact text for every entry is in `reports/project-audit-findings.csv`.

6. Missing mandatory requirements

Classification: **Mandatory** prevents a serious security, integrity or production-readiness failure; **Conditional** applies only if a feature, jurisdiction or deployment model applies; **Recommended** improves maintainability or operations. Status values: Implemented, Partial, Missing, Failed, Not verified. Legal compliance is not claimed or assessed.

ID	Class	Requirement	Status	Findings	Effort	Phase
M01	Mandatory	Every API endpoint authenticates the caller	Failed	API-001, API-002, API-003, SEC-017	M	0
M02	Mandatory	Server-side role and permission check on every write	Partial	SEC-001, SEC-002, SEC-005, SEC-006, PRV-001	M	1
M03	Mandatory	Tenant isolation derived from the session, never from process-global state	Failed	SEC-004, API-001, API-002, API-004, MNT-004	L	0
M04	Mandatory	Session revocation for deactivated users and suspended schools	Missing	SEC-003	M	1
M05	Mandatory	Durable brute-force and abuse protection	Partial	API-008, SEC-011, OPS-006, OBS-013	M	1
M06	Mandatory	Signed payment webhook plus reconciliation of pending payments	Missing	BIZ-002, OBS-007, FE-001, ARC-002	M	1
M07	Mandatory	Concurrency-safe payment recording with an overpayment cap	Failed	BIZ-003, DB-002, REL-001	M	1
M08	Mandatory	Scheduled jobs are authenticated and run for every tenant	Failed	BIZ-001, OBS-003, OBS-004, OPS-002, OBS-005	S	0
M09	Mandatory	Reproducible database migrations	Failed	DB-001, OPS-005	M	1
M10	Mandatory	Automated tests for authentication, authorization, tenant isolation and payments	Missing	TST-001, TST-002, MOB-007	L	1
M11	Mandatory	CI gates for lint, build and tests	Missing	OPS-001, TST-003, FE-003	M	1
M12	Mandatory	Patched framework and dependencies	Failed	SEC-008, SEC-009, SEC-015	S	0
M13	Mandatory	Identity documents are not publicly readable	Failed (suspected)	ARC-001, PRV-002, API-007	M	1
M14	Mandatory	No credentials in source; rotation of anything previously committed	Partial	SEC-014, DB-008	S	1
M15	Mandatory	Server-side input validation on every write route	Missing	API-005, API-006	L	1
M16	Mandatory	Security headers and a restricted CORS policy	Missing	SEC-007, OPS-007, SEC-019	S	1
M17	Mandatory	Error boundaries and non-leaky error responses	Missing	FE-002, ARC-004, OBS-002, API-009	S	2
M18	Mandatory	Audit trail for sensitive changes	Partial	OBS-008, BIZ-005	M	2
M19	Mandatory	Error tracking, structured logs and a health endpoint (admin app)	Missing	OBS-001, OPS-004, OBS-009	M	2
M20	Mandatory	Backups and a tested restore procedure	Not verified	OPS-005, OBS-010	S	1
M21	Mandatory	Environment variable validation and an example file	Missing	OPS-003	S	1
C01	Conditional	Children data protection: notice, consent, retention, erasure (if operating in India or serving minors)	Partial	PRV-003, PRV-004, PRV-005, PRV-006, PRV-008	L	2
C02	Conditional	Refund and reversal of payments (if online payments are live)	Missing	BIZ-005	M	2
C03	Conditional	Abuse limits on notification and bulk endpoints	Partial	API-006, SEC-016, BIZ-009, OBS-006	M	2
C04	Conditional	Production email provider	Partial	OBS-011	S	2
C05	Conditional	Accessibility basics (WCAG 2.2 A/AA)	Partial	UX-001, UX-002, MOB-008	M	3
C06	Conditional	Mobile session hygiene and release assets under version control	Partial	MOB-001, MOB-002, MOB-003, MOB-004, OPS-009	M	1
R01	Recommended	Paginate or aggregate remaining whole-roster loads	Partial	PERF-001, PERF-002, PERF-003, PERF-005, PERF-006, PERF-007	L	2
R02	Recommended	Index coverage and pool configuration	Partial	DB-004, PERF-004	M	3
R03	Recommended	Real foreign keys for class, section and subject	Missing	REL-004, DB-007, BIZ-007	XL	3

ID	Class	Requirement	Status	Findings	Effort	Phase
R04	Recommended	README, runbook and architecture documents	Missing	MNT-002	S	4

Requirement detail

M01 Every API endpoint authenticates the caller - Mandatory, Failed. Why it matters: Unauthenticated callers must never read or write tenant data. Risk if unresolved: Cross-tenant read/write, paid-fee and support-ticket tampering. Recommended implementation: Remove the session-less fallback actor, add a guard to the five unguarded handlers, deny by default in a shared wrapper. Evidence: API-001, API-002, API-003, SEC-017. Effort M, phase 0.

M02 Server-side role and permission check on every write - Mandatory, Partial. Why it matters: 101 of 180 routes use permissions; 29 only check that an actor exists; some pages have no guard. Risk if unresolved: A Parent or Accountant can write homework and marks, read full student records. Recommended implementation: Replace actor-non-null checks with requirePermission; add page-level guards; remove role shortcuts in libs. Evidence: SEC-001, SEC-002, SEC-005, SEC-006, PRV-001. Effort M, phase 1.

M03 Tenant isolation derived from the session, never from process-global state - Mandatory, Failed. Why it matters: The active school is a process-global singleton changed by every login. Risk if unresolved: Requests act on whichever school logged in last; roles editable across schools. Recommended implementation: Make resolveSchoolId session-only (error if absent); per-tenant roles; delete the singleton. Evidence: SEC-004, API-001, API-002, API-004, MNT-004. Effort L, phase 0.

M04 Session revocation for deactivated users and suspended schools - Mandatory, Missing. Why it matters: Disabled accounts stay valid until the 7-day JWT expires. Risk if unresolved: Fired staff and suspended schools keep access for a week. Recommended implementation: Check a token version or status on each request (cached), or shorten expiry with refresh tokens. Evidence: SEC-003. Effort M, phase 1.

M05 Durable brute-force and abuse protection - Mandatory, Partial. Why it matters: Limiter is in process memory, resets on cold start, XFF-spoofable. Risk if unresolved: Credential stuffing on serverless is effectively unthrottled. Recommended implementation: Move counters to a shared store (Redis/DB); key on account + IP from a trusted header. Evidence: API-008, SEC-011, OPS-006, OBS-013. Effort M, phase 1.

M06 Signed payment webhook plus reconciliation of pending payments - Mandatory, Missing. Why it matters: Payment finalisation depends on the browser calling verify. Risk if unresolved: Money captured but fee unpaid; disputes and manual reconciliation. Recommended implementation: Add a signature-verified idempotent webhook and a nightly reconcile of old PENDING payments. Evidence: BIZ-002, OBS-007, FE-001, ARC-002. Effort M, phase 1.

M07 Concurrency-safe payment recording with an overpayment cap - Mandatory, Failed. Why it matters: Read-modify-write outside the transaction; many PENDING orders per fee. Risk if unresolved: Double collection, overpayment, wrong balances. Recommended implementation: Do collection inside one transaction with a conditional update; cap payment at remaining due. Evidence: BIZ-003, DB-002, REL-001. Effort M, phase 1.

M08 Scheduled jobs are authenticated and run for every tenant - Mandatory, Failed. Why it matters: Crons skip auth when CRON_SECRET is unset and serve one school only. Risk if unresolved: Most schools never get generated fees; open endpoint can trigger jobs. Recommended implementation: Fail closed without the secret; iterate all active schools; report failures. Evidence: BIZ-001, OBS-003, OBS-004, OPS-002, OBS-005. Effort S, phase 0.

M09 Reproducible database migrations - Mandatory, Failed. Why it matters: One baseline migration covers 2 of 52 models. Risk if unresolved: A new environment cannot be built from migrations; drift is invisible. Recommended implementation: Baseline the full schema, run prisma migrate deploy in the release step. Evidence: DB-001, OPS-005. Effort M, phase 1.

M10 Automated tests for authentication, authorization, tenant isolation and payments - Mandatory, Missing. Why it matters: No test runner or test script in the admin app. Risk if unresolved: Every fix above can regress silently. Recommended implementation: Add a runner and integration tests for the critical flows first. Evidence: TST-001, TST-002, MOB-007. Effort L, phase 1.

M11 CI gates for lint, build and tests - Mandatory, Missing. Why it matters: No CI in either repository; lint baseline is red (97 errors). Risk if unresolved: Broken or vulnerable code can reach production unreviewed. Recommended implementation: Add a pipeline: lint, build, tests, npm audit; fix or baseline lint errors. Evidence: OPS-001, TST-003, FE-003. Effort M, phase 1.

M12 Patched framework and dependencies - Mandatory, Failed. Why it matters: npm audit reports 1 critical (next) and 13 high. Risk if unresolved: Known RCE/SSRF classes in the framework range in use. Recommended implementation: Upgrade next to the patched release; update nodemailer; re-run npm audit. Evidence: SEC-008, SEC-009, SEC-015. Effort S, phase 0.

M13 Identity documents are not publicly readable - Mandatory, Failed (suspected). Why it matters: Aadhaar and certificates sit under public, guessable URLs; bucket policy not verified. Risk if unresolved: Exposure of children and parent identity documents. Recommended implementation: Private bucket plus short-lived presigned GET; unguessable keys; migrate existing keys. Evidence: ARC-001, PRV-002, API-007. Effort M, phase 1.

M14 No credentials in source; rotation of anything previously committed - Mandatory, Partial. Why it matters: A password literal remains in a comment; seed holds shared passwords; history held a SuperAdmin credential. Risk if unresolved: Credential reuse if any value is live. Recommended implementation: Remove literals, rotate any live value, scan history with a secret scanner. Evidence: SEC-014, DB-008. Effort S, phase 1.

M15 Server-side input validation on every write route - Mandatory, Missing. Why it matters: zod schemas run only in client forms; malformed JSON gives 500. Risk if unresolved: Bad data, 500 errors, unbounded payloads. Recommended implementation: Validate bodies with the shared schemas in a route wrapper; cap array sizes. Evidence: API-005, API-006. Effort L, phase 1.

M16 Security headers and a restricted CORS policy - Mandatory, Missing. Why it matters: next.config is empty; CORS is * on all API routes. Risk if unresolved: Clickjacking, no CSP defence in depth. Recommended implementation: Add CSP, frame-ancestors, HSTS, nosniff; allow only known origins. Evidence: SEC-007, OPS-007, SEC-019. Effort S, phase 1.

M17 Error boundaries and non-leaky error responses - Mandatory, Missing. Why it matters: No error.jsx anywhere; err.message returned in about 97 places. Risk if unresolved: White-screen crashes; internal text leaks to clients. Recommended implementation: Add error.jsx/global-error.jsx; return generic messages and log the detail. Evidence: FE-002, ARC-004, OBS-002, API-009. Effort S, phase 2.

M18 Audit trail for sensitive changes - Mandatory, Partial. Why it matters: Only exams have an audit log; payments, roles and deletes do not. Risk if unresolved: No accountability for money or permission changes. Recommended implementation: Record actor, action, target and time for payments,

discounts, roles, deletes. Evidence: OBS-008, BIZ-005. Effort M, phase 2.

M19 Error tracking, structured logs and a health endpoint (admin app) - Mandatory, Missing. Why it matters: Sentry exists only in the mobile app. Risk if unresolved: Production failures are invisible. Recommended implementation: Add error tracking, request IDs, and a /health endpoint. Evidence: OBS-001, OPS-004, OBS-009. Effort M, phase 2.

M20 Backups and a tested restore procedure - Mandatory, Not verified. Why it matters: No documentation or evidence; provider settings not accessible. Risk if unresolved: Unrecoverable data loss. Recommended implementation: Confirm provider backups/PITR, document and rehearse a restore. Evidence: OPS-005, OBS-010. Effort S, phase 1.

M21 Environment variable validation and an example file - Mandatory, Missing. Why it matters: No validation; localhost URL fallback; no .env.example. Risk if unresolved: Silent misconfiguration in production. Recommended implementation: Validate required variables at startup; add .env.example. Evidence: OPS-003. Effort S, phase 1.

C01 Children data protection: notice, consent, retention, erasure (if operating in India or serving minors) - Conditional, Partial. Why it matters: Sensitive fields are plaintext; no erasure path; no consent capture. Legal compliance was not assessed. Risk if unresolved: Regulatory exposure. Recommended implementation: Define retention, add erasure/export, encrypt sensitive fields, publish a privacy notice. Evidence: PRV-003, PRV-004, PRV-005, PRV-006, PRV-008. Effort L, phase 2.

C02 Refund and reversal of payments (if online payments are live) - Conditional, Missing. Why it matters: permission key exists but no code path. Risk if unresolved: Manual database edits for refunds. Recommended implementation: Add a refund workflow with audit entries. Evidence: BIZ-005. Effort M, phase 2.

C03 Abuse limits on notification and bulk endpoints - Conditional, Partial. Why it matters: No per-user caps; fire-and-forget sends; token ownership transfer. Risk if unresolved: Spam, lost notifications. Recommended implementation: Cap recipients, await or queue sends, bind tokens to the verified user. Evidence: API-006, SEC-016, BIZ-009, OBS-006. Effort M, phase 2.

C04 Production email provider - Conditional, Partial. Why it matters: Without SMTP settings, mail is only logged to the console. Risk if unresolved: Invite and reset mails never arrive. Recommended implementation: Configure a provider; remove body logging. Evidence: OBS-011. Effort S, phase 2.

C05 Accessibility basics (WCAG 2.2 A/AA) - Conditional, Partial. Why it matters: No dialog semantics or focus trap; near-zero aria labels. Risk if unresolved: Unusable with keyboard or screen readers. Recommended implementation: Fix Modal and Dropdown first, then label controls. Evidence: UX-001, UX-002, MOB-008. Effort M, phase 3.

C06 Mobile session hygiene and release assets under version control - Conditional, Partial. Why it matters: Demo credentials on login; real app not under version control; keystore local only. Risk if unresolved: Credential exposure, unrecoverable signing keys. Recommended implementation: Remove demo box, commit the app to a remote, back up signing material. Evidence: MOB-001, MOB-002, MOB-003, MOB-004, OPS-009. Effort M, phase 1.

R01 Paginate or aggregate remaining whole-roster loads - Recommended, Partial. Why it matters: Several pages and jobs still load every row. Risk if unresolved: Slow pages and timeouts as data grows. Recommended implementation: Move to query-level pagination and DB aggregation, batch writes. Evidence: PERF-001, PERF-002, PERF-003, PERF-005, PERF-006, PERF-007. Effort L, phase 2.

R02 Index coverage and pool configuration - Recommended, Partial. Why it matters: No schoolId-leading indexes on several tables; no pool settings. Risk if unresolved: Slow queries and connection exhaustion under load. Recommended implementation: Add indexes from real query shapes; configure pooler and directUrl. Evidence: DB-004, PERF-004. Effort M, phase 3.

R03 Real foreign keys for class, section and subject - Recommended, Missing. Why it matters: Plain strings; local data already has orphan rows. Risk if unresolved: Drift and wrong reports after renames. Recommended implementation: Introduce ids with a backfill migration. Evidence: REL-004, DB-007, BIZ-007. Effort XL, phase 3.

R04 README, runbook and architecture documents - Recommended, Missing. Why it matters: README is boilerplate. Risk if unresolved: Slow onboarding, fragile operations. Recommended implementation: Write setup, env, deploy and recovery docs. Evidence: MNT-002. Effort S, phase 4.

7. Security assessment

OWASP Top 10 (2021) mapping, from `audit-reports/07-security-risk-register.md`:

Category	Status	Findings
A01 Broken access control	Weak	API-001..004, SEC-001, SEC-002, SEC-004..006, SEC-017, PRV-001
A02 Cryptographic failures	Mixed	PRV-003, SEC-010, API-010
A03 Injection	Good (no finding)	-
A04 Insecure design	Weak	SEC-004, API-001
A05 Security misconfiguration	Weak	SEC-007, SEC-019, OBS-012, API-009
A06 Vulnerable components	Failed	SEC-008, SEC-009, SEC-015
A07 Identification and authentication failures	Weak	SEC-003, SEC-010, SEC-011, SEC-013, SEC-018, API-008
A08 Software and data integrity failures	Weak	BIZ-002, FE-001, OPS-001
A09 Logging and monitoring failures	Weak	OBS-001, OBS-008
A10 SSRF	No server-side fetch of user URLs found; framework advisory applies	SEC-008

Dependency audit (`npm audit --json`, read-only): 14 findings, 1 critical (next, patched in a non-major release) and 13 high (nodemailer, prisma and eslint tool chains and transitive packages).

Role-permission matrix as seeded (roles are global and editable, see API-004):

Area	SuperAdmin	Principal	Admin	Teacher	Accountant	Parent
students view	X	X	X	X (scoped)	X	X (see PRV-001)
students create/update/delete	X	X	X	-	-	-

Area	SuperAdmin	Principal	Admin	Teacher	Accountant	Parent
teachers CRUD	X	X	X	-	-	-
attendance student view/mark	X	X	X	X (class teacher scope)	-	view only
attendance student update (lock)	X	X	X	-	-	-
leave apply / approve	X	X	approve+reject	apply	-	-
classes, subjects, timetable manage	X	X	X	timetable manage	-	-
exams manage / results publish	X	X	X	marks enter/update	-	results view
fees view/create/update/collect/refund	X	X	X	-	X	view, receipt
notices create/update/delete	X	X	X	X (own scope)	view	view
users view/create/update	X	X	X (no delete)	-	-	-
users delete	X	X	-	-	-	-
roles/permissions manage	X	X	-	-	-	-
settings update	X	X	X	-	-	-
performance reports	X	X	X	X (class teacher scope)	-	-

Note: Teacher holds timetable.manage (restricted to Class Teacher sections in the route). Matrix is the SEEDED default; roles are global and editable (API-004).

Route authorization coverage (180 route files, from `audit-reports/04-security-and-tenant-isolation.md`): most routes call `requirePermission`; 29 only check that an actor exists; 2 static template routes have no check; the five handlers in API-003 and `POST /api/schools` have none. The per-route table is in report 04.

8. Functional and technical audit results

- **Backend and business logic:** payments (races, no webhook, no refund), attendance (lock rules disabled in code, client-controlled fields), marks (edits after publish do not recompute), promotion (parallel transactions, no dues handling), notifications (fire-and-forget). IDs: T3, T12.
- **Database:** one baseline migration, plain-string references, JSON arrays of ids, string dates, missing tenant-leading indexes, inconsistent delete cascades. IDs: T7, T9, T8.
- **Performance and scalability:** whole-roster loads, per-student writes in single transactions, unbounded fan-outs, no pool configuration. Recent pagination and aggregation work removed several earlier hotspots. IDs: T8. No load test or EXPLAIN was run, so every at-scale statement is code-based.
- **Frontend and UX:** 97 lint errors, no error boundaries, no focus handling in Modal, partial keyboard support, mixed loaders. IDs: T13, T11.
- **Mobile:** demo credentials on the login screen, app not under version control, no 401 handling, push token not unregistered, no tests. IDs: MOB-001..012.
- **Observability and privacy:** no error tracking or audit trail in the admin app, plaintext Aadhaar, PAN and bank details, no erasure path. IDs: T4, T11.

9. Testing and verification summary

Check	Result	Notes
`npx eslint .` (admin)	Failed	exit 1; 710 files, 97 errors, 35 warnings; mostly set-state-in-effect (47) and no-unescaped-entities (42)
`npm audit --json` (admin)	Findings	1 critical, 13 high; read-only
`git ls-files` / `git check-ignore` (both repos)	Passed	`.env` files ignored and untracked; `dev.log` and four DB-touching scripts are tracked
Route inventory script (180 route files, comments stripped)	Passed	29 actor-non-null only; 2 without auth by design
Import graph script (703 files)	Passed	no static cycles
Read-only SELECTs for orphan counts	Passed	local database only, after a host check; no values printed
Live HTTP checks against endpoints	Blocked	dev server not running
`next build`, migrations, seed, mobile jest, load tests	Not run	forbidden by audit rules, or no tests exist

Tests that exist: none for the admin app; the mobile app has jest configured with zero app tests and a version mismatch (TST-002).

10. Prior audit reconciliation

The earlier audit (score 42/100) listed 67 claims. Re-checked against the current tree: 24 fixed, 7 partially fixed, 7 unverifiable (production environment), 29 still open. All 16 routes it named as unauthenticated mutation routes now begin with a guard. Detail in `audit-reports/14-prior-audit-reconciliation.md`.

11. Remediation roadmap

Phase 0 - Immediate containment

Task	Findings	Depends on	Effort	Acceptance criteria
Remove the session-less fallback actor and make resolveSchoolId session-only	API-001, API-002, SEC-004, SEC-017	none	M	An unauthenticated request to any /api route returns 401; no route reads or writes the global school singleton.
Add authentication to the five unguarded handlers (fee ledger, create-order, verify, attendance by student, subjects GET) and to POST /api/schools	API-002, API-003	none	S	Each handler returns 401/403 without a valid session; a test per route.
Fail closed when CRON_SECRET is unset; set it in every environment; iterate all active schools	BIZ-001, OBS-003, OBS-004, OPS-002	none	S	Cron without the bearer returns 401; a run generates fees for every active school.
Upgrade next to the patched release and re-run npm audit	SEC-008	none	S	npm audit shows no critical; app builds and logs in.

Phase 1 - Security, authorization, data integrity, production blockers

Task	Findings	Depends on	Effort	Acceptance criteria
Per-route permission checks for homework, marks, student records; guards on dashboard pages	SEC-001, SEC-002, SEC-005, SEC-006, PRV-001	Phase 0 wrapper	M	Parent/Accountant requests to those routes return 403; Teacher cannot render Teachers/Fees/Settings pages.
Tenant-scope roles and permissions; session revocation	API-004, SEC-003	Phase 0	L	Editing a role in one school never changes another; a deactivated user is rejected on the next request.
Payment webhook, reconcile job, transactional collection with cap	BIZ-002, BIZ-003, DB-002, OBS-007, FE-001, ARC-002	per-school webhook secret	M	Replayed webhook gives one payment; parallel collections cannot exceed due.
Private storage for identity documents and unguessable keys	ARC-001, PRV-002, API-007	object migration	M	Unauthenticated GET of a document URL returns 403.
Full-schema baseline migration and migrate deploy in release; confirm backups	DB-001, OPS-005, OPS-003	none	M	A fresh database builds from migrations; restore rehearsal documented.
Test runner plus integration tests for auth, tenancy, payments; CI pipeline	TST-001, OPS-001, TST-003, FE-003	none	L	CI runs lint, build, tests and audit on every pull request and is green.
Security headers, CORS allow-list, durable rate limit, server-side validation, credential cleanup	SEC-007, API-008, API-005, SEC-014, MOB-001, MOB-002, OPS-009, PRV-003	none	L	Headers present; limiter works across instances; invalid bodies return 400.

Phase 2 - Functional defects and reliability

Task	Findings	Depends on	Effort	Acceptance criteria
Error boundaries, generic error messages, error tracking and health endpoint	FE-002, ARC-004, OBS-002, API-009, OBS-001, OPS-004, OBS-009	Phase 1	M	A thrown render error shows the boundary; /health returns 200; errors reach the tracker.
Audit trail, refunds, notification delivery, business-rule fixes	OBS-008, BIZ-004, BIZ-005, BIZ-006, BIZ-008, BIZ-009, BIZ-010	Phase 1	L	Sensitive actions leave audit rows; marks edits after publish recompute results.
Remove whole-roster loads in classes, fees, bulk import, promotion	PERF-001, PERF-002, PERF-003, PERF-005, PERF-006	none	L	Measured response time and memory stay flat as rows grow (test listed in report 06).
Privacy: notice, consent, retention and erasure	PRV-004, PRV-005, PRV-006	legal input	L	Erasure request removes data and files; privacy notice linked.

Phase 3 - Performance, scalability, maintainability

Task	Findings	Depends on	Effort	Acceptance criteria
Indexes, pooling, caching of user and permissions	DB-004, PERF-004, PERF-008, PERF-011	none	M	EXPLAIN shows index use on listed queries; no connection exhaustion in a load test.
Real foreign keys for class, section, subject; remove duplicate sources of truth	REL-004, REL-005, DB-007, BIZ-007	backfill	XL	Renaming a class does not break history; orphan counts are zero.
Split god modules and share duplicated helpers	MNT-001, REL-007, REL-008	tests	L	Largest module below an agreed size; one formatCurrency.

Phase 4 - Testing, monitoring, documentation, ongoing hardening

Task	Findings	Depends on	Effort	Acceptance criteria
Accessibility pass	UX-001, UX-002, MOB-008	none	M	Keyboard-only walkthrough of core flows succeeds.
Docs, runbook, dead-code removal, tracked scratch files	MNT-002, ARC-005, OBS-012, OPS-008, TST-004	none	S	README covers setup, env, deploy, recovery; scratch files untracked.

Findings not named in a task above are scheduled by phase in the CSV (column RemediationPhase) and in the checklist: 62 of 144 entries.

12. Final recommendations

1. Fix identity and tenancy before anything else (Phase 0); every other control depends on it.
2. Add tests for authentication, authorization, tenancy and payments as you fix them, then CI to keep them green.
3. Finish the payment path: webhook, reconcile, transactional collection.
4. Protect identity documents and decide retention and erasure rules for children data.
5. Make the system reproducible and recoverable: full migrations, backups, runbook, error tracking.
6. Only then spend time on performance refactors, data-model clean-up and accessibility.

13. Not verified and limitations

- No live endpoint, role-by-role or tenant-isolation test was run; all security results are code-based.
- Production configuration was not accessible: Vercel settings, `CRON_SECRET` presence, storage bucket policy, database backups and pooling, EAS credentials, Pusher auth.
- Secret strength (`JWT_SECRET`, `ENCRYPTION_KEY`) was not assessed because values must not be read.
- Performance statements are from code inspection; no benchmarks were taken.
- Not every library function and component was read (about 40 of 166 `findMany` sites in context; the rest by pattern).
- Backend, database and observability streams stopped before exporting scores; their findings come from their reports.
- Counts changed between streams because the code changed; later streams re-verified open critical items.
- This report does not certify security or legal compliance.

14. Glossary

- **IDOR/BOLA:** access to another user's object by changing an id.
- **Tenant:** one school; tenant isolation means one school cannot reach another school's data.
- **RBAC:** role-based access control.
- **JWT:** signed session token.
- **Webhook:** server-to-server notification from the payment provider.
- **Idempotent:** repeating the same request has the same effect as doing it once.
- **Fail open / fail closed:** whether a missing safeguard allows or denies the action.

15. Report files

- `reports/project-audit-report.md` and `.pdf` - this report
- `reports/project-audit-findings.csv` - every finding with evidence and fix
- `reports/project-audit-requirements.csv` - requirement register
- `reports/project-audit-checklist.md` - remediation checklist
- `audit-reports/` - the fourteen area reports and machine-readable findings behind this report